Abstract

Abstract I

Research software repositories in monorepo configurations accumulate three categories of shared resources that individual projects must consume without re-implementing discovery logic: **data pools** (bibliographies, contacts, datasets), **governance rules** (style guides, coverage thresholds, citation schemas), and **executable tools** (code executors, validators, skill invocations). Without a canonical integration pattern, projects either duplicate discovery logic or silently ignore resources that fail to load — both outcomes degrade reproducibility and collaborative cohesion [@Wilson2014best;@Taschuk2017ten].

Abstract II

This paper presents `template_pools_rules_tools`, a meta-project exemplar that demonstrates how a single project can programmatically discover, validate, and exercise all three resource categories with zero tight coupling to any specific resource instance. The exemplar comprises eight Python modules — three resource readers (`fonds_reader`, `rules_applier`, `tools_invoker`), an orchestrator (`integration`), a semantic rule evaluator (`strong_rule_evaluator`), a figure generator (`figures`), a manuscript-token generator (`manuscript_variables`), and shared type definitions (`type_defs`) — plus six thin orchestration scripts and a fully token-injected manuscript pipeline.

Abstract III

The architecture (@fig:architecture) separates *resource ownership* from *resource consumption*. Resources live in top-level `fonds/`, `rules/`, and `tools/` directories and are never modified by consumers. Each resource exposes a typed manifest (`fonds.yaml`, `rules.yaml`, `tools.yaml`) that the corresponding reader module uses for discovery and validation. All readers implement graceful fallbacks: they return `None` or empty collections when a resource is absent, log a warning via the standard library `logging` module, and allow the integration pipeline to continue. This revision extends the original three-figure presentation to eight content figures plus a cover illustration — a fond taxonomy (@fig:taxonomy), a rule hierarchy (@fig:rulehier), a tool invocation contract (@fig:toolcontract), a three-level resilience diagram (@fig:resilience), and a script pipeline flow (@fig:pipelineflow) — so that every structural claim in the prose has a corresponding visual.

Abstract IV

In a representative pipeline run, the integration demo loaded 3 fonds, validated 2 rule sets, discovered 3 tools, and processed 8 bibliography entries — all reported as structured JSON that populates manuscript variable tokens at render time. Tests covering the eight `src/` modules (across nine test files) achieve well above the required 90% combined line coverage and use real file paths rather than mocks, ensuring that reported counts are genuine — run `uv run pytest … --cov-report=term` for the current test count and coverage percentage rather than trusting a number printed here.

The `template_pools_rules_tools` exemplar provides a reference implementation that any project in the template repository can consult when designing its own resource-consumption layer.

Abstract V

This reference implementation is deliberately reproducible end-to-end: every count, figure, and page-layout choice in this document is regenerated from source rather than hand-authored, per the Reproducibility Checklist in the front matter.

Keywords: research software engineering, monorepo architecture, reproducibility, fonds, governance rules, tool discovery, graceful degradation